

APPENDICE – Git e GitHub

Introduzione a Git

Il problema che risolve. La lezione parte dal caso banale: finché su un progetto ci lavoro **solo io, in locale, non c'è nessun problema**. Il problema nasce quando sullo stesso progetto lavorano cinque, dieci, cento sviluppatori. Due domande concrete:

1. **Come faccio a mantenere il codice aggiornato?** L'alternativa artigianale sarebbe: ognuno fa le sue modifiche e a fine giornata manda una mail a tutti. In un team di dieci persone vuol dire che ogni mattina devo aprire dieci mail, scaricare il sorgente e copiarlo nel mio progetto. *"Una guerra"*, dice — e soprattutto poco fattibile.
2. **E se io e un collega modifichiamo lo stesso file negli stessi punti?** Questa è la situazione che si chiama **conflitto**, e a mano è ingestibile.

Da qui nascono i **sistemi di controllo di versione (VCS)**: permettono a più persone di lavorare insieme sullo stesso codice e mantengono **più versioni** del progetto. L'esempio che usa: un progetto "casa vacanza" può avere la versione 1, la 1.2, la 1.3; posso avere una versione solo per gli sviluppatori, una solo per risolvere certi bug.

Di tool ce ne sono tanti — cita **CVS** e **SVN** — ma quello che negli ultimi anni si è imposto è **Git**.

I sei vantaggi di Git, nell'ordine in cui li elenca:

Vantaggio	Cosa significa davvero
Distribuito	ogni sviluppatore ha una copia locale completa del progetto. I VCS precedenti prevedevano un server centrale a cui collegarsi per ogni operazione; con Git puoi lavorare senza server centrale
Efficiente	le operazioni, in particolare i commit, sono molto veloci
Flessibile	si creano versioni parallele (branch) del progetto molto facilmente
Risolve i conflitti	molti li risolve da solo; per quelli importanti chiede allo sviluppatore quale riga tenere
Sicuro	usa algoritmi di crittografia per garantire l'integrità del codice
Ampia comunità	<i>"il successo o meno di un framework spesso dipende dalla facilità di reperire risorse e informazioni"</i> — più è diffuso, più trovi la soluzione al tuo caso
Integrazione	si integra con moltissimi strumenti, GitHub e GitLab in testa

Il punto sul "distribuito" merita una precisazione, perché è la cosa che distingue davvero Git da SVN e la lezione la sfiora soltanto: in Git **hai l'intera storia del progetto sul tuo disco**. Per questo `git log`, `git diff`, `git checkout` di un commit vecchio e la creazione di un branch funzionano **offline e istantaneamente** — non c'è nessuna chiamata di rete. In SVN quasi tutte queste operazioni contattavano il server.

Installazione. Si scarica da internet (`git-scm.com`), si sceglie la versione per il proprio sistema operativo e si eseguono i passaggi dell'installer. Per verificare, si apre il prompt (cmd su Windows) e si scrive:

```
git --version
```

Nella lezione lo sbaglia due volte scrivendo `git-version` col trattino prima di correggersi: è `git spazio --version`.

Configurazione dell'utente. Git per funzionare correttamente **ha bisogno di nome ed email**. Due comandi:

```
git config --global user.name "Antonio Pagano"
git config --global user.email "antonio@esempio.it"
```

Perché servono: nome ed email finiscono **dentro ogni commit** come firma dell'autore. Non sono credenziali, non autenticano nulla — sono metadati. Se li dimentichi, il primo `git commit` fallisce con un messaggio esplicito. `--global` significa "per tutti i repository di questo utente"; senza `--global`, la configurazione vale solo per il repository corrente (utile quando usi un'email di lavoro su alcuni progetti e una personale su altri).

Fondamenti di Git – repository, staging e commit

Il repository. È *"il posto in cui inseriremo tutti i nostri file, tutte le nostre modifiche"* — in concreto, la cartella che contiene il codice sorgente. Ci si posiziona nella cartella (nell'esempio C:\prova_git) e si scrive:

```
git init
```

Git risponde che ha inizializzato un repository vuoto e crea una cartella nascosta **.git**. Nella lezione deve andare in *Opzioni cartella → Visualizzazione → Mostra file nascosti* per vederla. Contiene i file di configurazione gestiti da Git: **non va assolutamente modificata a mano**.

Il ciclo add → commit. Crea un file `prova.txt` e vuole "inviarlo". Il flusso è in due tempi:

```
git add prova.txt
git commit -m "passo 1"
```

Git risponde che il commit è stato effettuato, con il messaggio "passo 1" e **1 file changed**.

Cos'è un commit. La definizione che dà, ed è quella giusta da tenere: *"una specie di **fotografia istantanea** del nostro codice, una specie di **punto di controllo**".* Tutto quello che ho fatto finora, lo salvo. Ogni commit è identificato da un **ID** univoco, e un repository ha tanti commit quante sono le istantanee che ho scattato.

Cos'è l'area di staging. `git add` non fa il commit: mette il file in un'area particolare chiamata **area di staging**, cioè l'insieme dei file **che verranno committati al prossimo commit**. Poi `git commit -m` prende ciò che sta nell'area di staging e crea l'istantanea.

I tre comandi di contorno che elenca:

```
git add .          # aggiunge tutti i file della cartella corrente
git reset nomefile # toglie un file dall'area di staging
git status         # mostra cosa è in staging e cosa no
```

Questo passaggio in due tempi è **la cosa che confonde di più chi viene da altri VCS**, e vale la pena avere in testa il quadro completo, che la lezione non disegna mai per intero:

```
Working directory --git add--> Staging area --git commit--> Repository locale
```

(i tuoi file) (l'index) (la storia)

Il senso dei due passaggi è poter **committare solo una parte** di quello che hai modificato: hai toccato cinque file ma solo tre riguardano il bug che stai chiudendo, quindi metti in staging quei tre.

Una nota di attualità: `git reset <file>` funziona ancora, ma dal **Git 2.23 (2019)** il comando pensato per questo è **`git restore --staged <file>`**, ed è quello che `git status` stesso ti suggerisce oggi.

La cronologia. Aggiunge un secondo file, `prova2.txt`, con `git add .` e `git commit -m "passo 2"`. E qui arriva il punto della lezione: *"ma perché tutta questa tarantella?"*. Perché nel repository **teniamo traccia di tutte le modifiche fatte**, e possiamo tornare indietro.

```
git log
```

mostra tutti i commit del repository, **dal più recente al più antico**, ognuno con il suo **ID**, l'**autore** e la **data**.

Tornare a un commit precedente. Si copia l'ID del commit desiderato e:

```
git checkout <id-del-commit>
```

Facendolo sul commit "passo 1", il file `prova2.txt` **scompare dalla cartella**: il codice è tornato allo stato in cui era. Ma Git avvisa: *"you are in **detached HEAD** state"*. La spiegazione della lezione: le modifiche che farai ora **non verranno riportate da nessuna parte**, perché il repository è "distaccato". Come farci qualcosa lo rimanda alla lezione sui branch.

Vale la pena capirlo davvero, perché è una situazione in cui si finisce spesso per sbaglio. **HEAD** è semplicemente un puntatore a "dove sei adesso". Normalmente punta a un **branch** (`master`), e il branch punta all'ultimo commit: quando committi, il branch avanza e ti porta dietro. Se fai checkout di un **ID di commit**, HEAD punta direttamente a quel commit, senza branch di mezzo: i commit che fai lì non sono agganciati a niente e quando torni indietro diventano irraggiungibili.

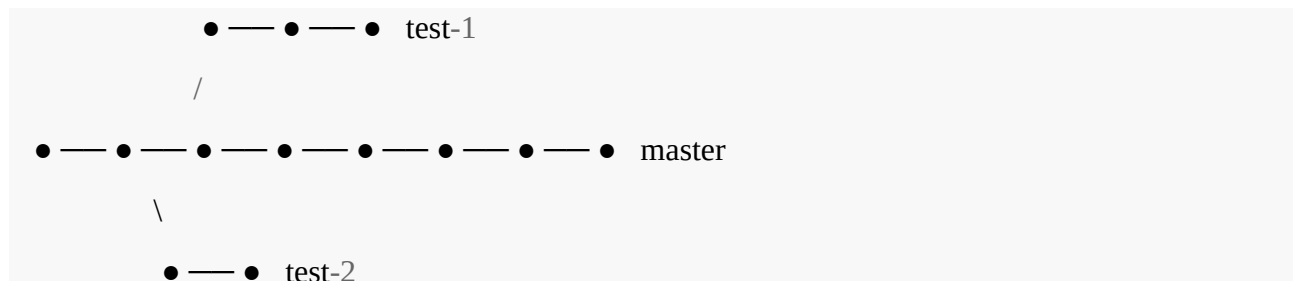
Per uscirne basta `git checkout master` (o `git switch -`). Se invece vuoi **davvero** lavorare da lì, la cosa giusta è creare subito un branch: `git switch -c nome-branch`.

Aggiungo un chiarimento che la lezione lascia ambiguo: `git checkout <commit>` **non cancella la storia**, ti sposta soltanto a guardarla da un altro punto. I comandi che modificano davvero la storia sono `git reset --hard` e `git revert`, e sono un'altra cosa.

i branch

Il modello mentale. Immagina il repository come una linea, e i pallini su questa linea come i commit. Quella linea è il **branch principale**, che Git crea di default e che *"in genere si chiama master"*.

Il branch è **un percorso parallelo di sviluppo**. La metafora che usa: se il codice è un albero, il branch principale è il **tronco** e i vari branch sono i **rami**.



Da un certo commit in poi si crea un nuovo ramo: le persone possono lavorare e committare lì mentre io continuo a lavorare su master. Si possono creare **quanti branch si vogliono, in qualsiasi punto**. A un certo punto un branch si può **unire** (merge) al master — oppure non unirlo mai, se si decide che quel lavoro non va bene.

A cosa servono, in pratica. I motivi che elenca:

- sviluppare una **nuova funzionalità** senza interferire con la linea principale: la sviluppo, la controllo, e **solo se va bene** la porto su master;
- fare **bug fixing** isolato;
- mantenere **versioni parallele**: la versione 2 in sviluppo e la versione 1 vecchia su cui continuo a fare piccole modifiche.

I comandi. Tutti sull'esempio della cartella `prova_git`:

```
git branch          # elenca i branch; l'asterisco verde è quello corrente
git branch test-2    # crea un nuovo branch
git checkout test-2   # ci si sposta sopra
git checkout -b test-3 # crea E si sposta, in un colpo solo
```

Cosa contiene un branch appena creato. Domanda esplicita della lezione, e la risposta è importante: contiene **il codice esattamente nel punto in cui l'abbiamo creato**. Nel momento della creazione, `test-1` è identico a `master`; da lì in poi le due linee divergono con i rispettivi commit.

Il merge, passo per passo. Sul branch `test-3` crea un file `provafile3.txt`, poi:

```
git add .
git commit -m "aggiunto file"
```

Ora `test-3` è diverso da `master`. Per unirlo:

```
git checkout master    # 1. ci si sposta sul branch di DESTINAZIONE
```

```
git merge test-3      # 2. si tira dentro il branch di origine
```

Il dettaglio didattico migliore della lezione: **prima di dare invio va a guardare la cartella**, e `provafile3.txt` non c'è, perché siamo su master. Dopo il merge, il file compare. Questo è il modo giusto di far vedere che i branch sono reali e che il contenuto della cartella cambia sotto i tuoi piedi.

La direzione del merge è la cosa da fissare: `git merge X` porta le modifiche di X **dentro il branch in cui ti trovi**. Chi sbaglia, sbaglia sempre qui.

Eliminare un branch. Una volta che `test-3` è stato riportato su master non serve più:

```
git branch -d test-3
```

Git conferma la cancellazione, e un `git branch` successivo mostra che è sparito.

Tre cose che la lezione non dice e che servono appena esci dall'esempio giocattolo:

1. master o main? Git crea ancora `master` in locale, ma **GitHub dal 2020 chiama main** il branch di default. Nella lezione successiva infatti su GitHub compare `main`, senza che venga mai spiegato il salto. Non è un errore, sono due default diversi. Si allinea con `git config --global init.defaultBranch main`.

2. -d contro -D. `git branch -d` è la versione **prudente**: rifiuta di cancellare un branch che non è ancora stato mergiato, proprio per non perdere lavoro. `-D` maiuscolo forza la cancellazione. Se ti dice *"the branch is not fully merged"*, sta facendo il suo lavoro: non forzare senza aver capito.

3. Il conflitto vero. La lezione 1 promette che Git gestisce i conflitti, ma qui il merge riesce sempre pulito perché i due branch toccano file diversi. Quando invece toccano **le stesse righe**, `git merge` si ferma e ti scrive dentro il file i marcatori:

```
<<<<<<< HEAD
```

```
la versione di master
```

```
=====
```

```
la versione di test-3
```

```
>>>>>>>test-3
```

Si apre il file, si tiene ciò che serve, **si cancellano le tre righe di marcatori**, poi `git add <file>` e `git commit`. Il 90% dei disastri da principiante è aver committato il file con i marcatori dentro.

4. checkout fa troppe cose. Ti sposta tra branch, tra commit, e ripristina file: tre lavori diversi con lo stesso nome, ed è il motivo per cui è così confuso. Dal Git 2.23 esistono i comandi separati: **git switch** per spostarsi (`git switch test-3`, `git switch -c nuovo`) e **git restore** per ripristinare file. `checkout` continua a funzionare e va benissimo saperlo, ma su codice nuovo conviene `switch`.

GitHub – Introduzione

Cos'è e perché serve. GitHub è "un servizio online molto utilizzato tra gli sviluppatori che permette di conservare, condividere e collaborare". L'argomento con cui lo introduce: se su un progetto siamo tantissime persone, non è che ci scambiamo il progetto **con la chiavetta**. Serve un **server centralizzato** a cui mandare tutte le modifiche. Quel posto è GitHub.

GitHub mette a disposizione dei **repository** — una cartella in cui mettere tutto il progetto — e per ognuno mantiene la cronologia e segnala i conflitti.

Registrazione e creazione di un repository. Ci si registra (versione gratuita, con una versione a pagamento per le aziende con più funzionalità). Nella dashboard, a sinistra ci sono i propri repository; si clicca **New Repository** e si compila:

Campo	Nella lezione
Owner	l'account
Repository name	prova-rep
Description	una descrizione libera
Public / Private	pubblico

Add a README file sì

Sul **pubblico** fa una precisazione che vale la pena non perdere: pubblico significa che **chiunque può vedere** il repository, ma **chi può modificarlo lo decido io** — le persone vanno aggiunte esplicitamente al progetto.

Il **README** è "il classico file di testo in cui inseriremo le istruzioni per far girare il nostro progetto": informazioni sul prodotto, configurazioni da fare.

Gli strumenti di GitHub. Passa in rassegna le tre schede principali:

Issue. "Tutti i progetti hanno dei bug, ne escono sempre". Le issue sono il posto dove queste problematiche vengono tracciate. Una issue ha titolo, descrizione, **label** e può essere **assegnata a una persona**. Nella demo non può assegnarla perché il repository non ha ancora membri.

Pull request. "Stiamo suggerendo delle modifiche che possiamo fare al progetto". Non modifico direttamente il progetto: faccio le modifiche su una copia e dico "guarda, ho fatto queste modifiche". Poi sta al proprietario del repository accettarle e unirle al codice principale, oppure rifiutarle.

Settings → Collaborators. Dove si aggiungono i collaboratori del progetto.

Il flusso di lavoro che tiene insieme le tre cose — e questa è la frase più importante della lezione:

"Io creo l'issue e la assegno a una persona. Questa persona, quando ha risolto l'issue, crea una pull request. Poi io verifico se effettivamente risolve quell'issue e faccio il merge con il ramo principale."

ISSUE	→	BRANCH + COMMIT	→	PULL REQUEST	→	MERGE
(il problema)		(la soluzione)		(la proposta)		(l'accettazione)

assegnata a X scritta da X aperta da X fatto dal team leader

Chiude con l'ovvietà che però è vera: *"non esiste un programmatore che non conosce GitHub"*, è utilizzato in tutte le software house.

Due note di contesto. **Primo:** GitHub non è l'unico — esistono **GitLab** e **Bitbucket**, che offrono lo stesso modello (su GitLab le pull request si chiamano *merge request*). **Secondo:** dal 2020 anche i piani gratuiti includono i **repository privati illimitati**, quindi la scelta pubblico/privato oggi non è più una questione di prezzo. E su un repository pubblico ricorda che è pubblico **anche tutto lo storico**: una password committata per sbaglio e poi rimossa resta visibile nei commit precedenti.

Clonare un repository GitHub

Il prerequisito: essere collaboratore. Prima di poter *modificare* un repository bisogna essere autorizzati. Il team leader va su **Settings** → **Collaborators** e aggiunge la persona. La lezione precisa: se sei tu che hai creato il repository non devi fare niente, ma è bene sapere che questo passaggio esiste.

Clonare. Sul repository si clicca **Code**, si copia l'URL, ci si posiziona nella cartella dove si vuole scaricare e:

```
git clone <url-del-repository>
```

Git copia il repository remoto in locale, creando **una cartella con lo stesso nome del repository**, con dentro i file identici al remoto — nell'esempio, solo il README.

Il ciclo completo locale → remoto. Crea un file di testo e lo vuole mandare sul remoto:

```
cd prova-rep
git add .
git commit -m "primo"
git push
```

Dopo il push, ricaricando la pagina GitHub il file compare, **con accanto il messaggio del commit**.

Il riepilogo, che è il punto della lezione:

Fino a `git commit` le modifiche sono **solo nel repository locale**. Per mandarle sul repository remoto serve **git push**.

Comando	Dove agisce
<code>git add</code>	working directory → staging
<code>git commit -m</code>	staging → repository locale
<code>git push</code>	repository locale → repository remoto

"Ogni volta che entrerete in un progetto nuovo dovrete clonare il repository remoto".

Qui c'è il buco più grosso di tutta l'appendice, e va colmato: la lezione non nomina mai **git pull**. Il flusso è presentato a senso unico, ma se lavori in team **gli altri**

pushano mentre tu lavori, e il tuo push verrà rifiutato con *"updates were rejected because the remote contains work that you do not have locally"*. La soluzione non è forzare, è:

```
git pull    # scarica le modifiche del remoto e le unisce alle tue
git push    # ora passa
```

`git pull` è in realtà due comandi in uno: **git fetch** (scarica cosa c'è sul remoto, senza toccare il tuo lavoro) + **git merge** (lo unisce). La regola pratica da tenere: **git pull prima di iniziare a lavorare la mattina, e prima di ogni push.**

Due cose collegate che mancano e che servono da subito su un progetto Java:

.gitignore — un file di testo nella radice del repository con l'elenco di ciò che Git deve ignorare. Su un progetto Maven è obbligatorio, altrimenti committi la cartella `target/` compilata:

```
target/
*.class
.idea/
.settings/
*.iml
```

origin — è semplicemente il **nome di default che Git dà al repository remoto** da cui hai clonato. Nella lezione successiva compare `git push origin . . .` senza che sia mai spiegato: non è una parola magica, è un'etichetta (`git remote -v` te lo mostra).

Pull request

Cos'è. Spesso abbreviata **PR**, è *"un modo per proporre modifiche da un branch all'altro"*. In pratica è una domanda: *"ho fatto queste modifiche al codice sorgente, le accetti o no?"*. Il gestore del repository risponde accettando e facendo il merge, oppure rifiutando.

Come si inserisce nel lavoro reale. Il team leader assegna le issue ai programmatori — **una issue = un bug o un compito**. Ogni issue diventa **una pull request a sé stante**: il programmatore, quando ha finito il compito assegnato, crea la PR e la invia al team leader. Sta poi al team leader accettarla o meno.

Il flusso completo, comando per comando.

```
# 1. Creo un branch locale per la mia issue
git checkout -b issue-180

# 2. Faccio le mie modifiche (nell'esempio aggiunge un file)

# 3. Le committo in locale
git add .
git commit -m "risolto issue 180"
```

4. Mando IL BRANCH sul repository remoto

```
git push origin issue-180
```

Ricaricando GitHub compaiono ora **due branch**: **main** (quello attivo, con il codice corrente) e **issue-180**.

Aprire la pull request. Due modi, entrambi mostrati:

- Git stesso, dopo il push, stampa nel terminale un **link diretto** per creare la PR per quel branch: si copia e si incolla nel browser;
- oppure si va sulla scheda **Pull requests** → **New pull request**.

Si mette il titolo ("*Risolto issue 180*") e si clicca **Create pull request**.

E qui la lezione dice la cosa più importante:

"Per quanto riguarda noi sviluppatori, abbiamo finito. Stop. Punto."

Da lì in poi tocca al team leader: si collega, apre la PR, fa le verifiche, e **se non ci sono conflitti la può mergiare direttamente da GitHub**. Dopo il merge la PR si chiude, *"e dovrei chiudere anche l'issue"*. Andando su **Code** si vedono ora sia le modifiche che c'erano prima, sia quelle della PR appena accettata.

Il flusso di lavoro standard, come lo riassume lui:

1. agguirno il mio repository locale
2. creo un nuovo branch
3. faccio le mie modifiche
4. commit
5. `git push origin <mio-branch>`
6. apro la pull request
7. il team leader decide: merge oppure no

Alcune precisazioni utili sul mondo reale, perché questo è l'argomento che incontrerai davvero al lavoro:

- **Il passo 1 è git pull** — che, come detto, la lezione non nomina mai. Se salti l'aggiornamento, il tuo branch parte da una base vecchia e la PR arriverà piena di conflitti.
- **git push -u origin <branch>** con la -u la prima volta: collega il branch locale a quello remoto, così dalle volte successive basta `git push` liscio.
- **La PR non è solo un bottone, è il posto della code review.** Su GitHub si commenta riga per riga, si chiedono modifiche (*Request changes*), si approva. È lì che, nella pratica, avviene metà del lavoro di un team.
- Se scrivi nella descrizione della PR **Closes #180**, GitHub **chiude automaticamente l'issue 180** quando la PR viene mergiata — risolve esattamente il *"dovrei chiudere anche l'issue"* della lezione.
- Nel mondo reale la PR fa anche **partire i test automatici** (GitHub Actions): se falliscono, il bottone di merge si blocca. È il ponte tra questo argomento e la CI.

- Se non sei collaboratore del repository (progetti open source), il flusso cambia leggermente: prima si fa **fork** (una copia del repository sul tuo account), poi la PR parte dal tuo fork.